

Process Memory Monitoring and Security Detection System Using Object-Oriented C++ Design

Dita Salsabila Lubis

Student

Department of Computer Engineering
Kastamonu university, Kastamonu, Turkey
Email: ditasalsabilalubis@gmail.com

Abstract—In modern computing environments, system security and resource monitoring are essential for maintaining stability and detecting malicious or abnormal behavior. Intrusion detection systems play a crucial role in cybersecurity frameworks by identifying suspicious activities within a system [2]. This paper presents a simplified Process Memory Monitoring and Security Detection System developed using Object-Oriented Programming (OOP) principles in C++. The system simulates process monitoring, memory evaluation, and anomaly detection based on predefined thresholds. The proposed architecture demonstrates modular design using encapsulation, class interaction, and separation of responsibilities. Although simplified, the system reflects core concepts used in operating system tools and security software. The C++ programming language is widely used for system-level applications due to its performance and low-level memory control capabilities [3].

Keywords—Process Monitoring, Memory Usage, Security Detection, C++, Object-Oriented Programming, Intrusion Detection System

I. INTRODUCTION

Modern operating systems require efficient mechanisms to monitor processes and manage system resources. Without proper monitoring, systems may suffer from memory leaks, performance degradation, or malicious exploitation.

Intrusion detection systems (IDS) were first formally modeled by Denning as a method to detect unauthorized access and abnormal behavior in computer systems [2]. These systems form the foundation of modern cybersecurity tools such as antivirus software and endpoint protection systems.

Process monitoring is a fundamental concept in operating systems where resources such as CPU and memory are allocated and managed by the kernel [4]. Process monitoring tools such as Task Manager in Windows or top in Linux provide real-time insights into system performance. These tools help users understand memory consumption and CPU usage across running processes.

This project proposes a simplified simulation of such systems using C++ and Object-Oriented Programming (OOP), focusing on:

- Process tracking
- Memory usage monitoring
- Rule-based anomaly detection
- Alert generation

The C++ language is selected due to its efficiency and suitability for system-level programming [3].

II. RELATED WORK

Process monitoring and intrusion detection systems have been widely studied in cybersecurity research.

Axelsson proposed a taxonomy of intrusion detection systems, categorizing them into anomaly-based and signature-based detection methods [1]. Anomaly-based systems detect deviations from normal behavior, while signature-based systems rely on known patterns of attacks.

Denning's model introduced the concept of statistical anomaly detection for identifying suspicious activities in computer systems [2]. These foundational works influence modern security tools used in enterprise environments.

C++ remains a popular choice for implementing performance-critical system tools due to its ability to directly manage memory and system resources efficiently [3].

II. SYSTEM DESIGN

A. Design Overview

The system design is inspired by core operating system principles such as process management and resource allocation [4].

The system consists of four main components:

- Process
- MemoryMonitor
- SecurityDetector
- SystemManager

Each component is implemented as a separate class following OOP principles.

B. UML Conceptual Design

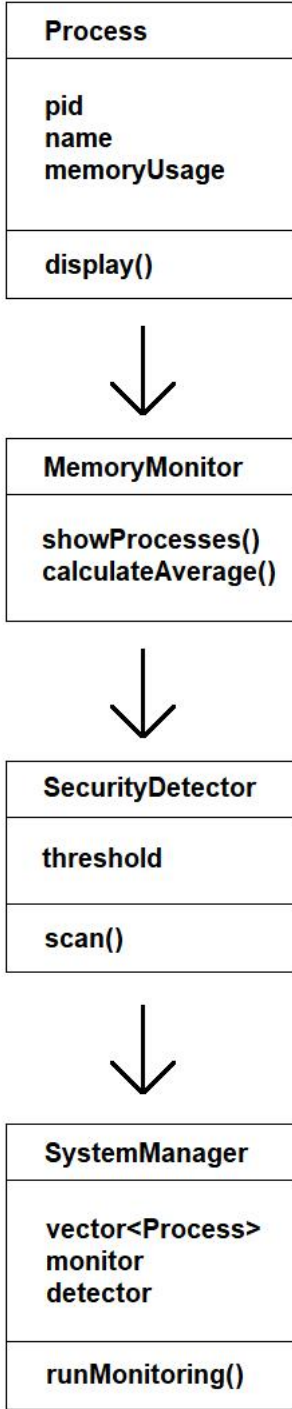


Fig. 1. System architecture of the proposed process monitoring and security detection system.

Fig. 1 illustrates the overall system architecture of the proposed monitoring tool. The system consists of four main components: Process, MemoryMonitor, SecurityDetector, and SystemManager. Each component is implemented as a separate class following object-oriented design principles.

C. Class Descriptions

1) Process Class

Encapsulates process information such as PID, name, and memory usage. This ensures data hiding and modularity.

2) MemoryMonitor Class

Responsible for:

- Displaying process data
- Calculating average memory usage across processes

3) SecurityDetector Class

Implements a rule-based detection system. Any process exceeding a defined threshold is flagged as suspicious.

4) SystemManager Class

Acts as the controller of the system, managing processes and coordinating monitoring and detection operations.

IV. IMPLEMENTATION

The system is implemented in C++ using standard libraries such as `<iostream>` and `<vector>`. Core detection logic:

```

if(p.getMemoryUsage() > threshold) {
    cout << "[ALERT] Suspicious
    Process Detected!";
}
  
```

This logic represents a simple form of anomaly detection based on threshold comparison.

V. ALGORITHM

Algorithm: Process Memory Monitoring and Detection

1. Initialize system with a memory threshold
2. Create process list
3. Add processes to system
4. Display all processes
5. Compute average memory usage
6. For each process:
 - If memory usage > threshold → trigger alert
7. End process scanning

VI. VI. RESULTS AND DISCUSSION

A. Simulation Output

```
--- Process Memory Report ---
PID: 1 | Name: chrome.exe | Memory: 450 MB
PID: 2 | Name: explorer.exe | Memory: 120 MB
PID: 3 | Name: game.exe | Memory: 800 MB
PID: 4 | Name: notepad.exe | Memory: 50 MB

Average Memory Usage: 355 MB

--- Security Scan Report ---
[ALERT] Suspicious Process Detected!
PID: 1 | Name: chrome.exe | Memory: 450 MB
[ALERT] Suspicious Process Detected!
PID: 3 | Name: game.exe | Memory: 800 MB
```

B. Analysis

The system demonstrates:

- ✓ Encapsulation of process data
- ✓ Modular OOP architecture
- ✓ Basic security detection logic
- ✓ Separation of responsibilities

However, it also has limitations:

- No real OS-level process integration
- No dynamic system monitoring
- No advanced AI-based detection
- Fixed rule-based threshold system only

VII. LIMITATIONS

This system is a simulation and does not interact with real operating system processes. Real-world systems such as antivirus software and intrusion detection systems operate at kernel or driver level [2].

Additionally, professional tools often combine rule-based and statistical anomaly detection methods for better accuracy [1].

VIII. FUTURE WORK

Future improvements include:

1. System-Level Integration
Using OS APIs for real process monitoring.
2. Advanced Detection Systems
Machine learning-based anomaly detection instead of fixed thresholds.
3. Real-Time Monitoring
Multithreading for continuous scanning.
4. Logging System
Saving alerts into external log files.
5. GUI Development
Using frameworks like Qt for visualization.

C++ remains suitable for these enhancements due to its efficiency and system-level control capabilities [3].

IX. CONCLUSION

This paper presented a Process Memory Monitoring and Security Detection System using Object-Oriented Programming in C++. The system successfully simulates core functionalities of security monitoring tools, including process tracking, memory analysis, and anomaly detection.

Although simplified, the implementation demonstrates how software engineering principles can be applied to model real-world cybersecurity systems. The study also highlights the importance of OOP design in building scalable and maintainable software systems.

ACKNOWLEDGMENT

The author would like to express gratitude to the course instructor and Kastamonu University for providing the academic environment and resources necessary for the completion of this project. Appreciation is also extended to everyone that is contributed to the implementation of the proposed system.

REFERENCES

- [1] P. Axelsson, "Intrusion Detection Systems: A Survey and Taxonomy," 2000.
- [2] D. E. Denning, "An Intrusion-Detection Model," IEEE Transactions on Software Engineering, 1987.
- [3] B. Stroustrup, The C++ Programming Language, Addison-Wesley.
- [4] Silberschatz, Galvin, Gagne, Operating System Concepts, Wiley.